

Vorlesung

Theoretische Informatik

(10 Anwendung kontextfreier Sprachen: Evaluation arithmetischer Ausdrücke)

Alois Heinz
Hochschule Heilbronn,
Max-Planck-Str. 39, 74081 Heilbronn
heinz@hs-heilbronn.de

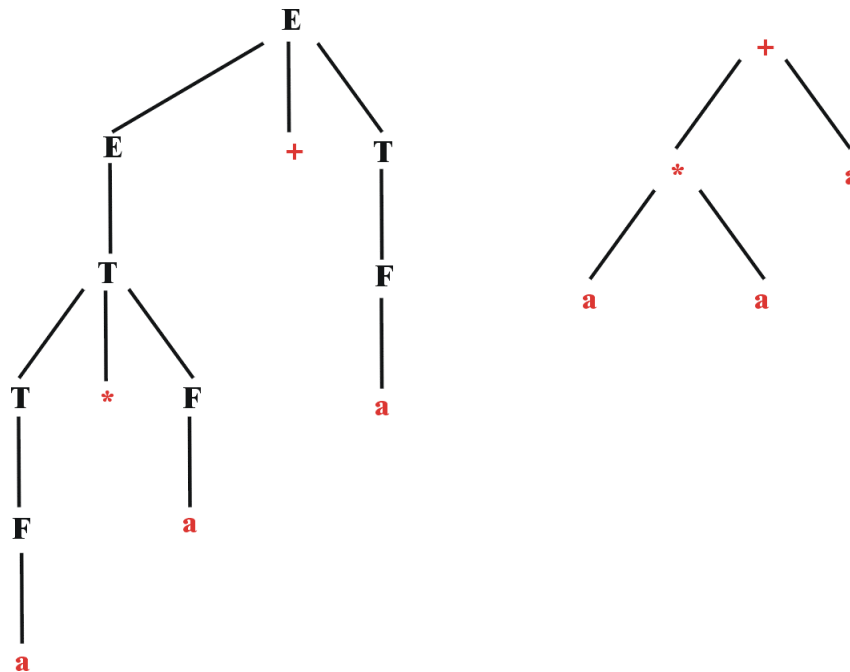
12. Mai/19. Juni 2026

Eindeutige Grammatik für Arithmetische Ausdrücke

$$P = \{E \rightarrow E + T \mid E - T \mid T, T \rightarrow T * F \mid T / F \mid F, F \rightarrow (E) \mid a\}$$

Jedes Wort $w \in L(G_{Arith}^{eind})$ kann nur durch **einen Ableitungsbaum** (eine Linksableitung | eine Rechtsableitung) abgeleitet werden.

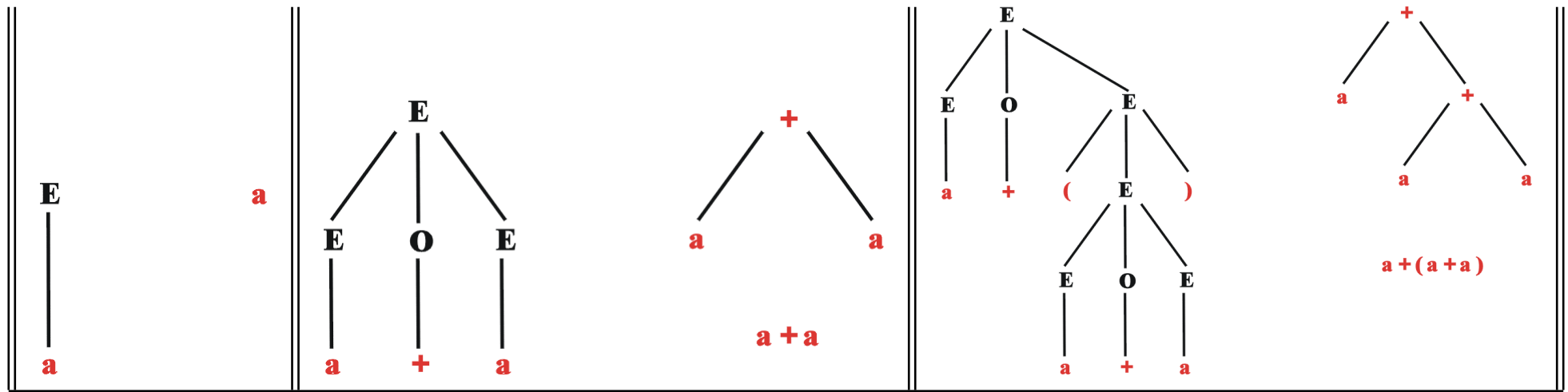
Jedem Ableitungsbaum entspricht eindeutig ein **Termbaum** (Syntax-, Ausdrucks-, Kantorovic-Baum, Operator-Baum):
Innere Knoten enthalten Operator-Symbole, Blätter enthalten die Operanden.



Transformation Ableitungsbaum $\rightarrow$ Termbaum

1. Jedes Blatt mit Markierung „(“ oder „)“ kann gestrichen werden
2. Jeder innere Knoten mit genau einem Nachfolger wird durch diesen ersetzt
3. Bei jedem inneren Knoten k mit 3 Nachfolgern wird der mittlere Nachfolger gestrichen und sein Operatorsymbol als Markierung in k übertragen.

Beispiel für die Grammatik G_{Arith} :



Auswertung eines Termbaums

Die **Auswertung** (Evaluation) eines Termbaums kann erfolgen durch Aufruf einer **rekursiven** Methode für die **Wurzel**:

1. Ist der aktuelle Knoten ein Blatt, dann wird die Markierung als Wert interpretiert (geparst).
2. Ist der aktuelle Knoten ein innerer Knoten, so werden linker und rechter Teilbaum evaluiert und anschließend die Operation aus der Markierung angewandt.

```
double eval () {  
    double val = 0.0;  
    if (isLeaf()) val = Double.parseDouble (mark);  
    else switch (mark.charAt(0)){  
        case '+' : val = left.eval() + right.eval(); break;  
        case '-' : val = left.eval() - right.eval(); break;  
        case '*' : val = left.eval() * right.eval(); break;  
        case '/' : val = left.eval() / right.eval(); break;  
    }  
    return (val);  
}
```

Umbau der Regeln

Die Regeln der Grammatik G_{Arith}^{eind}

$$P = \{E \rightarrow E + T \mid E - T \mid T, \\ T \rightarrow T * F \mid T / F \mid F, \\ F \rightarrow (E) \mid a\}$$

eignen sich noch nicht so sehr zur effizienten Termbaumerstellung:

Die **links-rekursiven Regeln** stören und werden daher umgebaut:

$$P' = \{E \rightarrow TX, \\ X \rightarrow \epsilon \mid + TX \mid - TX, \\ T \rightarrow FY, \\ Y \rightarrow \epsilon \mid * FY \mid / FY, \\ F \rightarrow (EZ \mid a, \\ Z \rightarrow)\}$$

Die erzeugbaren Ausdrücke können auch so beschrieben werden:

$$E \rightarrow T \{\{+ \mid -\} T\}^*, \\ T \rightarrow F \{\{* \mid /\} F\}^*, \\ F \rightarrow (E) \mid a$$

Rekursiver Aufbau eines Termbaums (1)

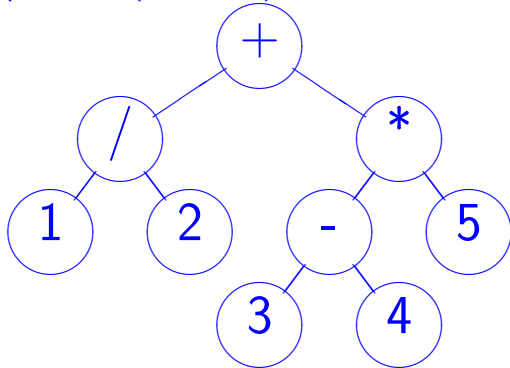
```
public static Atree expression () throws Exception {
    Atree a = term ();
    while (hasNextChar() && (nextChar() == '+' || nextChar() == '-')){
        switch (getNextChar()){
            case '+' : a = new Atree ("+", a, term ()); break;
            case '-' : a = new Atree "-", a, term ()); break;
        }
    }
    return a;
} // expression

public static Atree term () throws Exception {
    Atree a = factor ();
    while (hasNextChar() && (nextChar() == '*' || nextChar() == '/')){
        switch (getNextChar ()){
            case '*' : a = new Atree ("*", a, factor ()); break;
            case '/' : a = new Atree ("/", a, factor ()); break;
        }
    }
    return a;
} // term
```

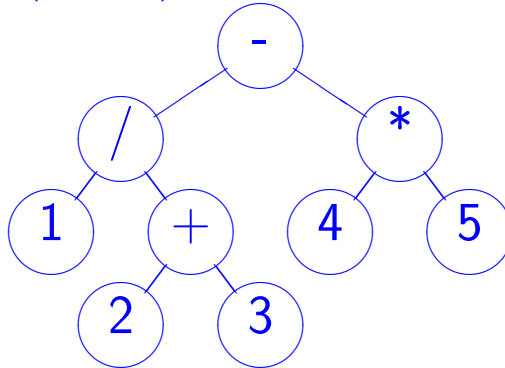
Rekursiver Aufbau eines Termbaums (2)

```
public static Atree factor () throws Exception {  
    if (!hasNextChar()) throw new Exception ("in expr, end of input");  
    return nextChar () == '(' ? bracedexpression () : number ();  
} // factor  
  
public static Atree bracedexpression () throws Exception {  
    getNextChar();  
    Atree a = expression ();  
    if (!(hasNextChar() && getNextChar()==''))  
        throw new Exception ("in bracedexpression, \")\" expected");  
    return a;  
} // bracedexpression
```

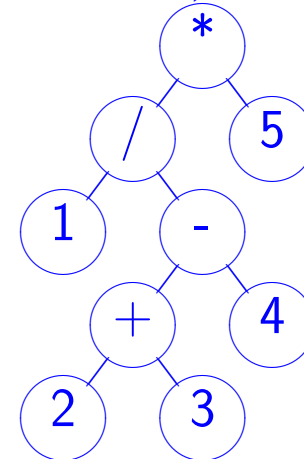
$$1/2 + (3 - 4) * 5 = -4.5$$



$$1/(2 + 3) - 4 * 5 = -19.8$$



$$1/(2 + 3 - 4) * 5 = 5.0$$



Iterative Auswertung von Ausdrücken mit Keller (1)

```
public static double eval () throws Exception {
    while (!ops.isEmpty())
        switch (ops.pop ()) {
            case E : ops.push (X).push (T); break;
            case T : ops.push (Y).push (F); break;
            case X : if (hasNextChar() && (nextChar() == '+' || nextChar() == '-')) {
                        ops.push (X);
                        switch (getNextChar()) {
                            case '+' : ops.push (ADD); break;
                            case '-' : ops.push (SUB); break;
                        }
                        ops.push (T);
                    } break;
            case Y : if (hasNextChar() && (nextChar() == '*' || nextChar() == '/')) {
                        ops.push (Y);
                        switch (getNextChar()) {
                            case '*' : ops.push (MUL); break;
                            case '/' : ops.push (DIV); break;
                        }
                        ops.push (F);
                    } break;
        }
```


Iterative Auswertung von Ausdrücken mit Keller (2)

```
case ADD: vals.push ( vals.pop()+vals.pop());      break;
case SUB: vals.push (-(vals.pop()-vals.pop()));      break;
case MUL: vals.push ( vals.pop()*vals.pop());      break;
case DIV: vals.push ( (1.0/vals.pop())*vals.pop()); break;
case F : if (!hasNextChar())
            throw new Exception ("in factor, end of input");
            if (nextChar () == '(') {
                getNextChar();
                ops.push (Z).push(E);
            } else vals.push (number());
            break;
case Z : if (!(hasNextChar() && getNextChar()=='))
            throw new Exception ("in expr, \"\")\" expected");
            break;
} // switch, while
return vals.pop ();
} // eval
```

Zwei Keller werden benutzt: `ops` für die Ableitung, `vals` für die berechneten (Zwischen-) Werte. Mit `nextChar` wird das nächste zu lesende Eingabezeichen untersucht, das den nächsten Ableitungsschritt eindeutig bestimmt.

Direkte rekursive Auswertung von Ausdrücken (1)

```
public static double expression () throws Exception {
    double val = term ();
    while (hasNextChar() && (nextChar() == '+' || nextChar() == '-'))
        switch (getNextChar()){
            case '+' : val += term (); break;
            case '-' : val -= term (); break;
        }
    return val;
} // expression

public static double term () throws Exception {
    double val = factor ();
    while (hasNextChar() && (nextChar() == '*' || nextChar() == '/'))
        switch (getNextChar ()){
            case '*' : val *= factor (); break;
            case '/' : val /= factor (); break;
        }
    return val;
} // term
```

Direkte rekursive Auswertung von Ausdrücken (2)

```
public static double factor () throws Exception {
    if (!hasNextChar()) throw new Exception ("in expr, end of input");
    return nextChar () == '(' ? bracedexpression () : number ();
} // factor

public static double bracedexpression () throws Exception {
    getNextChar();
    double val = expression ();
    if (!(hasNextChar() && getNextChar()==''))
        throw new Exception ("in bracedexpression, \")\" expected");
    return val;
} // bracedexpression
```

Zusammenfassung: Auswertung von Ausdrücken

- Voraussetzung: Grammatik, die eindeutig ist und den jeweils nächsten Links-Ableitungsschritt mit Hilfe des nächsten Eingabezeichens eindeutig macht
- Aufbau des Termbaums und Evaluation des Termbaums, oder
- Iterative Auswertung mit Hilfe von Kellerspeichern für Ableitung und Werte, oder
- Direkte rekursive Auswertung